Probabilistic Robotics

Mihir Rao

Based on Probabilistic Robotics by Thrun, Burgard & Fox

Contents

1 Introduction **2**
1.1 Uncertainty in robotics . . . 2
1.2 Beliefs and the hallway example . . . 2
1.3 Uncertainty and action . . . 3
1.4 Benefits and costs . . . 4
1.5 Scope and background . . . 4

2 Recursive State Estimation **5**
2.1 Basic probability tools . . . 5
2.2 Robot and environment interaction . . . 7
2.3 Beliefs and the Bayes filter . . . 8
2.4 Worked example: the door . . . 10
2.5 Assumptions and implementation . . . 10

3 Gaussian Filters **12**
3.1 Representing a Gaussian belief . . . 12
3.2 The linear Gaussian model . . . 12
3.3 The Kalman filter update . . . 13
3.4 Worked example: one-dimensional tracking . . . 15
3.5 The extended Kalman filter . . . 15
3.6 The information filter . . . 17
3.7 The extended information filter . . . 19
3.8 Computational and modeling tradeoffs . . . 20

4 Nonparametric Filters **21**
4.1 Discrete Bayes filters . . . 21
4.2 Histograms for continuous states . . . 21
4.3 Static binary state and log odds . . . 23
4.4 Particle beliefs and the bootstrap filter . . . 25
4.5 Why importance weights work . . . 25
4.6 Systematic resampling . . . 27
4.7 Failure modes and resource choices . . . 29

Chapter 1: Introduction

A robot acts in a physical world that it can observe only partially. Its measurements, movements, and models are imperfect. Probabilistic robotics represents that uncertainty explicitly and uses it to interpret observations and choose actions.

1.1 Uncertainty in robotics

A factory can tightly control the placement of objects and the tasks a robot performs. A home or road is less predictable. The book identifies five sources of uncertainty:

1. **The environment.** Objects and people move, and the robot cannot predict every change.
2. **Sensors.** Measurements have noise, limited range, and limited resolution. Different situations can produce similar readings.
3. **Actuation.** A commanded motion is not an exact displacement. Wheel slip and control errors change the result.
4. **Models.** A mathematical model approximates the physical world and omits some of its detail.
5. **Computation.** Decisions must be made in time, often using approximations that can be computed quickly.

Better hardware can reduce some errors, but uncertainty still has to be handled in the robot's reasoning.

1.2 Beliefs and the hallway example

Definition 1.1 [Belief]. A belief is a probability distribution over possible states, given the information available to the robot. It can preserve several competing explanations of the same observations.

In localization, the robot estimates its position relative to a map. In global localization, its starting position is unknown. Consider the book's hallway with three similar doors. The task is to determine which door the robot is near, using observations and movement together.

1. **Before observing.** With no reason to favor one hallway position, the robot starts with a uniform belief over the allowed positions.

2. **After seeing a door.** Positions near all three doors become more likely. The reading cannot distinguish the doors, so the belief has three peaks. Other positions retain some probability because the sensor can err.

3. **After moving.** Each location hypothesis moves by the estimated displacement. Motion uncertainty spreads the peaks.

4. **After another door reading.** The robot favors hypotheses that explain both the displacement and the new observation. A distinctive spacing between doors can make one hypothesis much more likely than the others.

Keeping the whole belief allows later evidence to resolve the ambiguity. Choosing one door immediately would discard alternatives before there was enough information to distinguish them.

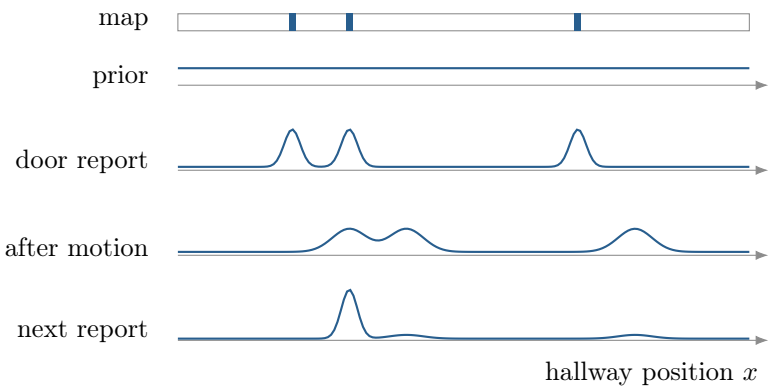


Figure 1: The hallway example, drawn schematically. A door reading creates three possible locations. Motion shifts and spreads the peaks; another door reading favors the hypothesis consistent with the spacing between doors.

1.3 Uncertainty and action

Perception estimates the state from data. Action changes both the state and the information the robot can obtain. A robot may therefore choose a movement partly because it will make its location easier to determine.

- Exploration gathers information.
- Exploitation uses current knowledge to carry out the task.

A decision can depend on the competing hypotheses in the belief, rather than only its most likely state. The robot must balance information gathering with progress toward its goal.

1.4 Benefits and costs

The probabilistic approach supports several related tasks:

- **Robust localization:** combine uncertain motion and sensor data, and recover when the robot's estimated location is wrong.
- **Mapping:** estimate the environment even without a reliable global position reference.
- **Learning:** estimate persistent unknown quantities from data collected over time.

These capabilities depend on suitable models and algorithms. Their main computational cost is representing and updating a distribution. Compact representations make the computation practical, but limit which kinds of uncertainty the robot can retain.

1.5 Scope and background

The book develops state-estimation methods first, then motion and measurement models, localization and mapping, and finally decisions under uncertainty. Algorithms are studied through their derivations, procedures, implementation issues, and experimental behavior.

Classical, reactive, and hybrid approaches differ in how they combine models, planning, and immediate responses to sensing. Probabilistic methods make uncertainty part of that reasoning. Randomized search is a different use of probability: using random numbers to search does not itself represent uncertainty about the world's state.

Source. Thrun, Burgard, and Fox, Probabilistic Robotics, early draft, Chapter 1, pp. 1–8 (PDF pp. 13–20).

Chapter 2: Recursive State Estimation

State estimation infers quantities that are not directly observable from sensor data. The result is a belief over possible states. This chapter sets up the probability notation, models the robot's interaction with its environment, and derives the recursive Bayes filter.

2.1 Basic probability tools

2.1.1 Distributions and moments

Write X for a random variable and x for a value it can take. The book abbreviates $P(X = x)$ as $p(x)$.

- **Discrete state.** The probabilities are nonnegative and satisfy $\sum_x p(x) = 1$. For a fair coin with outcomes H, T, $p(H) = p(T) = 1/2$.
- **Continuous state.** The density is nonnegative and satisfies $\int p(x)\,dx = 1$. Probabilities are areas:
$$P(a \le X \le b) = \int_a^b p(x)\,dx.$$
A density's height can exceed one; its total area must equal one.

The EXPECTATION is a probability-weighted average. When the moments exist,

$$\mu = \mathbb{E}[X] = \sum_x xp(x) \quad \text{or} \quad \mu = \int xp(x)\,dx, \qquad \mathbb{E}[aX + b] = a\mathbb{E}[X] + b.$$

The COVARIANCE measures how deviations from the mean vary together:

$$\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^T].$$

Its diagonal entries are variances. In one dimension, $\sigma^2 = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mu^2$.

A scalar GAUSSIAN has mean μ, variance σ^2, and density

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x - \mu)^2}{2\sigma^2}\right].$$

For a d-dimensional vector, the corresponding density is

$$\mathcal{N}(x; \mu, \Sigma) = \frac{\exp[-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)]}{(2\pi)^{d/2} \det(\Sigma)^{1/2}}, \qquad \Sigma \succ 0.$$

Here μ is a vector and Σ is a symmetric positive-definite matrix.

For a discrete distribution, ENTROPY measures average uncertainty:

$$H(X) = -\sum_x p(x)\log_2 p(x), \qquad 0\log_2 0 := 0.$$

For the fair coin above, $H(X) = -2(\frac{1}{2}\log_2\frac{1}{2}) = 1$ bit. If the outcome is certain, entropy is zero. The book later uses entropy to measure information gained through a robot's actions.

2.1.2 Conditioning and Bayes' rule

A JOINT DISTRIBUTION describes two variables together. A CONDITIONAL DISTRIBUTION describes one when the other is known:

$$p(x,y) = p(x \mid y)p(y), \qquad p(x \mid y) = \frac{p(x,y)}{p(y)}.$$

The ratio is defined where $p(y) > 0$. Marginalizing over Y gives the LAW OF TOTAL PROBABILITY:

$$p(x) = \sum_y p(x \mid y)p(y) \quad \text{or} \quad p(x) = \int p(x \mid y)p(y)\,dy.$$

This averages the conditional answers over the possible values of Y.

Writing the joint distribution in both orders, $p(x,z) = p(x \mid z)p(z) = p(z \mid x)p(x)$, gives BAYES' RULE:

$$p(x \mid z) = \frac{p(z \mid x)p(x)}{p(z)} = \eta\, p(z \mid x)p(x).$$

- $p(x)$ is the PRIOR, before incorporating the reading z.
- $p(z \mid x)$ is the LIKELIHOOD: how probable that reading would be if the state were x.
- $p(x \mid z)$ is the POSTERIOR, after incorporating z.

The normalizer $\eta = 1/p(z)$ is the same for every candidate state x. Compute the products $p(z \mid x)p(x)$ first, then divide by their sum or integral to make the posterior total one.

2.1.3 Independence and conditional independence

Variables are INDEPENDENT when

$$p(x,y) = p(x)p(y), \qquad \text{equivalently} \qquad p(x \mid y) = p(x).$$

Knowing Y then does not change the distribution of X. CONDITIONAL INDEPENDENCE makes the corresponding statement after another variable S is known:

$$X \perp Y \mid S \quad \Longleftrightarrow \quad p(x, y \mid s) = p(x \mid s)p(y \mid s).$$

Neither ordinary independence nor conditional independence implies the other in general.

The door example gives a useful interpretation. Let D be the door's state, open or closed, and let Z_1, Z_2 be camera reports taken while that state stays unchanged. The model treats the reports as independent once D is known:

$$p(z_1, z_2 \mid D) = p(z_1 \mid D)p(z_2 \mid D).$$

When D is unknown, however, the first report changes the belief about the door. That changes the prediction for the second report. Thus two reports can be related overall even though they are conditionally independent given the state that produced them.

2.2 Robot and environment interaction

The state includes the quantities needed to predict what happens next. It may contain robot pose, joint configurations and velocities, objects in a map, moving people, or sensor health. Some variables are static; others change over time. A state with both continuous and discrete variables is called HYBRID.

Notation	Meaning
x_t	State at time t
u_t	Control information for the interval $(t-1, t]$
z_t	Measurement associated with state x_t
$u_{1:t}, z_{1:t}$	Control and measurement histories through t

Controls describe state changes; measurements provide information about a state. Odometry is physically measured, but is treated as control information because it describes a change in pose. The control-then-measurement ordering is a model of the data; real motion and sensing can occur together.

2.2.1 The Markov assumptions

A COMPLETE STATE makes earlier history unnecessary for predicting the future. The model uses two conditional-independence assumptions:

$$p(x_t \mid x_{0:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t \mid x_{t-1}, u_t),$$
$$p(z_t \mid x_{0:t}, z_{1:t-1}, u_{1:t}) = p(z_t \mid x_t).$$

- The TRANSITION MODEL $p(x_t \mid x_{t-1}, u_t)$ describes how state changes under a control.
- The MEASUREMENT MODEL $p(z_t \mid x_t)$ describes the readings a state can produce.

Together with the initial distribution $p(x_0)$, these laws define the probabilistic model used by the filter.

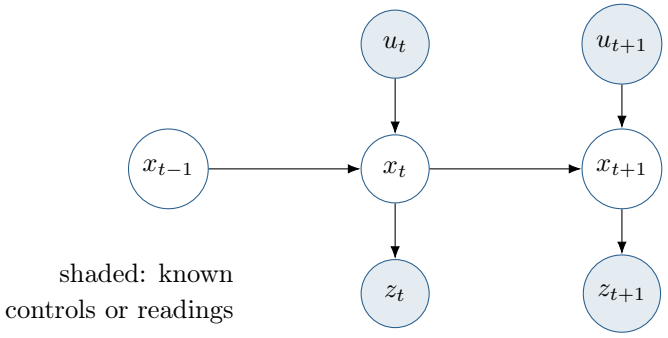


Figure 2: The transition and measurement dependencies. Given the current state, earlier history adds no information about its measurement.

2.3 Beliefs and the Bayes filter

Write $b_t(x_t) = \mathrm{bel}(x_t)$ for the belief after the latest reading, and use a bar for the prediction before that reading:

$$b_t(x_t) = p(x_t \mid z_{1:t}, u_{1:t}),$$
$$\bar{b}_t(x_t) = p(x_t \mid z_{1:t-1}, u_{1:t}).$$

The BAYES FILTER alternates two steps:

1. **Prediction.** Average the possible transitions over the previous belief:

$$\bar{b}_t(x_t) = \int p(x_t \mid x_{t-1}, u_t) b_{t-1}(x_{t-1})\, dx_{t-1}. \tag{2.1}$$

2. **Correction.** Reweight the prediction by the likelihood of the new reading and normalize:

$$b_t(x_t) = \eta\, p(z_t \mid x_t)\bar{b}_t(x_t). \tag{2.2}$$

Prediction accounts for uncertain motion. Correction favors states that better explain the sensor data.

2.3.1 Derivation

Let $\mathcal{H}_{t-1} = (z_{1:t-1}, u_{1:t-1})$. Treat the controls as given inputs that provide no further information about the previous state once this recorded history is known.

1. **Separate the latest reading with Bayes' rule.**

$$p(x_t \mid \mathcal{H}_{t-1}, u_t, z_t) = \eta\, p(z_t \mid x_t, \mathcal{H}_{t-1}, u_t)p(x_t \mid \mathcal{H}_{t-1}, u_t).$$

   The measurement assumption reduces the first factor to $p(z_t \mid x_t)$. The second factor is the prediction $\bar{b}_t(x_t)$, proving correction.

2. **Expand the prediction over the previous state.**

$$\bar{b}_t(x_t) = \int p(x_t \mid x_{t-1}, \mathcal{H}_{t-1}, u_t)p(x_{t-1} \mid \mathcal{H}_{t-1}, u_t)\, dx_{t-1}.$$

   The transition assumption reduces the first factor to $p(x_t \mid x_{t-1}, u_t)$. The assumption about controls reduces the second to $b_{t-1}(x_{t-1})$, proving prediction.

3. **Apply the identities recursively.** Starting from the correct prior b_0, the two steps give the correct posterior at each time by induction.

Algorithm 2.1: One Bayes-filter update

Input: Previous belief b_{t-1}, control u_t, reading z_t
for each candidate state x_t **do**
  $\bar{b}_t(x_t) \leftarrow \int p(x_t \mid x_{t-1}, u_t)b_{t-1}(x_{t-1})\, dx_{t-1}$
  $b_t(x_t) \leftarrow p(z_t \mid x_t)\bar{b}_t(x_t)$
Normalize b_t so that its integral is one
return b_t

For discrete state, replace integrals by sums. Initialize b_0 from the available knowledge: a point mass for a known state, a localized distribution for approximate knowledge, or a uniform distribution over a bounded feasible region for global uncertainty. Normalization requires positive total weight.

2.4 Worked example: the door

The door is either open (O) or closed (C). Initially both states have probability $1/2$. The camera can report either state incorrectly:

	$D = O$	$D = C$
$p(\text{report open} \mid D)$	0.6	0.2
$p(\text{report closed} \mid D)$	0.4	0.8

Waiting leaves the door unchanged. Pushing leaves an open door open and opens a closed door with probability 0.8. The goal is to update the belief after waiting and seeing "open", then pushing and seeing "open" again.

1. **Wait, then incorporate the first report.** The prediction remains $(1/2, 1/2)$. Multiplying by the "open" likelihoods gives weights

   $$(0.6 \cdot 0.5,\ 0.2 \cdot 0.5) = (0.3, 0.1).$$

   Their sum is 0.4, so

   $$b_1(O) = 0.3/0.4 = 3/4, \qquad b_1(C) = 1/4.$$

2. **Predict after the push.** The door can be open because it was already open, or because the push opened it:

   $$\bar{b}_2(O) = 1(3/4) + 0.8(1/4) = 0.95, \qquad \bar{b}_2(C) = 0.2(1/4) = 0.05.$$

3. **Incorporate the second report.** The new weights are $(0.6 \cdot 0.95, 0.2 \cdot 0.05) = (0.57, 0.01)$, whose sum is 0.58. Thus

   $$b_2(O) = \frac{57}{58} \approx 0.983, \qquad b_2(C) = \frac{1}{58}.$$

The final belief strongly favors an open door. It still retains a closed-door possibility, which can matter when acting on a wrong estimate would be costly.

2.5 Assumptions and implementation

A practical state rarely contains everything that can affect future data. For example, a localization state may describe the robot's pose but omit moving people who affect its readings. The book identifies several sources of error:

- Unmodeled environmental dynamics.
- Inaccurate transition or measurement models.

- Approximate representations of the belief.
- Persistent controller variables omitted from the modeled state.

Adding state variables can improve the model but increases computation. The aim is to choose a manageable state whose omitted influences are close to random under the chosen models.

Implementations of the Bayes filter must also balance:

- **Efficiency:** how much computation each update needs.
- **Accuracy:** which shapes of belief the representation can retain.
- **Ease of implementation:** which integrations, likelihoods, or sampling operations the models permit.

The general continuous-state algorithm is a specification. The next two chapters give finite representations that make it practical to compute.

Source. Thrun, Burgard, and Fox, Probabilistic Robotics, early draft, Chapter 2, pp. 9–32 (PDF pp. 21–44).

Chapter 3: Gaussian Filters

Gaussian filters represent the belief by a mean and covariance. The Kalman filter computes this belief exactly for linear Gaussian models. The extended Kalman filter uses the same representation with local approximations for nonlinear models. Information filters store the same Gaussian differently.

3.1 Representing a Gaussian belief

For a d-dimensional state, write the belief as $\mathcal{N}(x; \mu, \Sigma)$.

- The mean μ is a d-component vector.
- The symmetric covariance Σ is a $d \times d$ matrix. Diagonal entries describe coordinate variances; off-diagonal entries describe how coordinate errors vary together.
- Mean and covariance determine the full Gaussian distribution. Storing them requires $O(d^2)$ space.

This is the MOMENTS REPRESENTATION. A single Gaussian has one mode, so it is useful for tracking a state already known approximately. It cannot preserve several separated location hypotheses, as in global localization.

3.2 The linear Gaussian model

The Kalman filter assumes

$$x_t = A_t x_{t-1} + B_t u_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_t), \tag{3.1}$$

$$z_t = C_t x_t + \delta_t, \qquad \delta_t \sim \mathcal{N}(0, Q_t). \tag{3.2}$$

Here R_t is *motion* noise covariance and Q_t is *measurement* noise covariance, following the book's notation. For a d-dimensional state, m-dimensional control, and k-dimensional measurement:

$$A_t : d \times d, \quad B_t : d \times m, \quad C_t : k \times d, \quad R_t : d \times d, \quad Q_t : k \times k.$$

Three conditions keep the belief Gaussian:

1. The initial belief is Gaussian.
2. State transitions are linear with additive Gaussian noise.
3. Measurements are linear with additive Gaussian noise.

The noise terms are independent of the preceding state and of each other, with the corresponding independence across time. Controls are known inputs.

3.3 The Kalman filter update

Given $(\mu_{t-1}, \Sigma_{t-1})$, perform the following calculation.

1. **Predict through the motion model.**

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t, \tag{3.3}$$
$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t. \tag{3.4}$$

   The mean follows the nominal motion. The covariance is transformed by A_t on both sides because it describes products of coordinate deviations, and then gains the independent motion-noise covariance.

2. **Compare the reading with its prediction.**

$$r_t = z_t - C_t \bar{\mu}_t, \qquad S_t = C_t \bar{\Sigma}_t C_t^T + Q_t, \tag{3.5}$$
$$K_t = \bar{\Sigma}_t C_t^T S_t^{-1}. \tag{3.6}$$

   The INNOVATION r_t is the discrepancy in measurement coordinates. S_t is its covariance, including both state uncertainty projected into the sensor and the sensor's own noise. The KALMAN GAIN K_t converts this discrepancy into a state correction.

3. **Correct the belief.**

$$\mu_t = \bar{\mu}_t + K_t r_t, \tag{3.7}$$
$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t. \tag{3.8}$$

   The covariance update accounts for information supplied by the measurement. It must use the same predicted covariance that was used to compute the gain.

For a scalar direct measurement ($C_t = 1$), with predicted variance P and measurement variance Q,

$$K = \frac{P}{P+Q}, \qquad \mu = (1-K)\bar{\mu} + Kz, \qquad \sigma^2 = \frac{PQ}{P+Q}.$$

A small measurement variance Q gives the reading more weight. A small predicted variance P gives the prediction more weight. With $P, Q > 0$, the combined variance is smaller than either input variance because the two independent sources constrain the same unknown.

3.3.1 Why these equations implement Bayes filtering

Prediction. An affine transformation of a Gaussian, plus independent Gaussian noise, is Gaussian. Taking expectation in (3.1) gives (3.3). Subtracting that mean gives

$$x_t - \bar{\mu}_t = A_t(x_{t-1} - \mu_{t-1}) + \varepsilon_t.$$

Multiply this expression by its transpose and take expectation. The cross terms vanish by independence and zero noise mean, leaving $A_t\Sigma_{t-1}A_t^T + R_t$. This proves the prediction covariance as well.

Correction. Suppress time subscripts and let $P = \bar{\Sigma}$. For this derivation, take P and Q to be positive definite. Multiplying the predicted Gaussian by the likelihood gives

$$b(x) \propto \exp[-J(x)], \qquad J(x) = \tfrac{1}{2}(x-\bar{\mu})^T P^{-1}(x-\bar{\mu}) + \tfrac{1}{2}(z-Cx)^T Q^{-1}(z-Cx).$$

This is a quadratic in x, so the posterior is Gaussian. Its curvature and stationary point determine its covariance and mean:

$$\Lambda := P^{-1} + C^T Q^{-1} C = \Sigma^{-1}, \tag{3.9}$$

$$\Lambda\mu = P^{-1}\bar{\mu} + C^T Q^{-1} z. \tag{3.10}$$

Subtract $\Lambda\bar{\mu}$ in the second equation to obtain

$$\mu - \bar{\mu} = \Sigma C^T Q^{-1}(z - C\bar{\mu}).$$

This already has the desired innovation form. To recover the computationally useful gain and covariance, let $S = CPC^T + Q$. Then

$$\Lambda PC^T = C^T Q^{-1} S, \qquad \text{hence} \qquad \Sigma C^T Q^{-1} = PC^T S^{-1} = K.$$

The corresponding inversion identity is

$$\Sigma = P - PC^T(CPC^T + Q)^{-1}CP = (I - KC)P. \tag{3.11}$$

To verify it, multiply its right-hand side by Λ:

$$\Lambda(P - PC^T S^{-1} CP) = I + C^T Q^{-1} CP - C^T Q^{-1} S S^{-1} CP = I.$$

The identity gives the stated covariance update while requiring a solve in measurement space, rather than inversion of the full state covariance.

3.4 Worked example: one-dimensional tracking

Suppose a robot's location has mean 10 meters and variance 4 square meters. It commands a 2-meter displacement, with motion-noise variance 1. A direct position sensor then reports 14 meters with noise variance 4. These numbers are illustrative.

1. **Predict.** With $A = B = C = 1$,

$$\bar{\mu} = 10 + 2 = 12, \qquad P = 4 + 1 = 5.$$

2. **Calculate the discrepancy and gain.**

$$r = 14 - 12 = 2, \qquad S = 5 + 4 = 9, \qquad K = 5/9.$$

3. **Correct.**

$$\mu = 12 + (5/9)2 = \frac{118}{9} \approx 13.111, \qquad \sigma^2 = (1 - 5/9)5 = \frac{20}{9} \approx 2.222.$$

The reading pulls the estimate rightward, while the predicted uncertainty prevents treating 14 as exact. At the next time step, these posterior parameters become the previous belief; they are not replaced by the raw sensor reading.

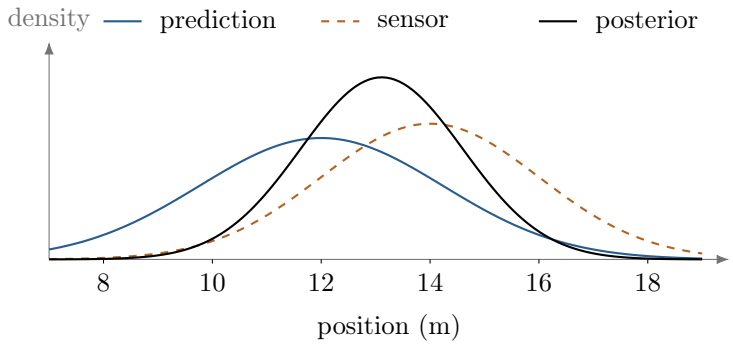


Figure 3: The worked example's Gaussian densities. The posterior is narrower and lies between the prediction and sensor reading. The sensor curve is the direct-measurement likelihood viewed as a function of position.

3.5 The extended Kalman filter

Real motion and sensing are often nonlinear:

$$x_t = g(u_t, x_{t-1}) + \varepsilon_t, \qquad z_t = h(x_t) + \delta_t.$$

For example, moving forward in the plane involves $\cos\theta$ and $\sin\theta$. Passing a Gaussian through such a function generally does not produce another Gaussian. The EXTENDED KALMAN FILTER (EKF) uses the function's tangent near the current estimate.

3.5.1 Linearize about the correct means

First-order Taylor expansion gives

$$g(u_t, x) \approx g(u_t, \mu_{t-1}) + G_t(x - \mu_{t-1}),$$
$$h(x) \approx h(\bar{\mu}_t) + H_t(x - \bar{\mu}_t),$$

where the JACOBIANS contain partial derivatives:

$$G_t = \left.\frac{\partial g(u_t, x)}{\partial x}\right|_{\mu_{t-1}}, \qquad H_t = \left.\frac{\partial h(x)}{\partial x}\right|_{\bar{\mu}_t}.$$

The motion Jacobian is evaluated at the previous mean, while the measurement Jacobian is evaluated at the *predicted* mean. Controls are fixed inputs when differentiating with respect to state.

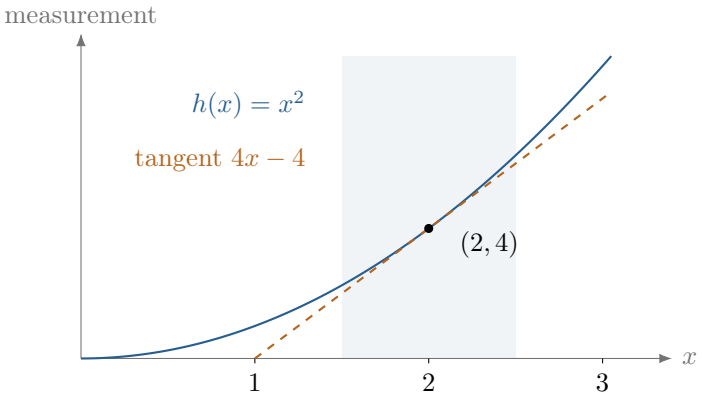


Figure 4: The tangent matches the function's value and slope at the mean. It is a local approximation: the discrepancy grows away from $x = 2$. The shaded band is an illustrative neighborhood, not a confidence region.

Substitute the affine approximations into the linear Gaussian calculation:

$$\bar{\mu}_t = g(u_t, \mu_{t-1}), \qquad \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t, \tag{3.12}$$
$$S_t = H_t \bar{\Sigma}_t H_t^T + Q_t, \qquad K_t = \bar{\Sigma}_t H_t^T S_t^{-1}, \tag{3.13}$$
$$\mu_t = \bar{\mu}_t + K_t[z_t - h(\bar{\mu}_t)], \qquad \Sigma_t = (I - K_t H_t)\bar{\Sigma}_t. \tag{3.14}$$

Use the nonlinear functions to predict the mean and reading; use the Jacobians to propagate their local uncertainty. Replacing the predicted reading by $H_t\bar{\mu}_t$ would lose the affine offset and generally

be wrong. The derivation is the Kalman derivation applied to these local affine models, so the result is exact for the approximations, not for the original nonlinear system.

3.5.2 Worked example: a nonlinear measurement

Suppose the predicted scalar state has mean 2 and variance 1/4. The sensor measures $h(x) = x^2$ with noise variance 1, and reports $z = 5$. The goal is to approximate the posterior after this reading. First evaluate the predicted reading and its derivative at the predicted mean:

$$H = 2\bar{\mu} = 4, \quad h(\bar{\mu}) = 4, \quad S = 4^2(1/4) + 1 = 5, \quad K = \frac{(1/4)4}{5} = \frac{1}{5}.$$

The EKF gives

$$\mu = 2 + \frac{1}{5}(5 - 4) = \frac{11}{5}, \qquad \sigma^2 = (1 - \tfrac{1}{5} \cdot 4)\frac{1}{4} = \frac{1}{20}.$$

The new mean is 2.2 and the new variance is 0.05. The reading was above the predicted value $h(2) = 4$, so the local update moves the mean upward.

3.5.3 Accuracy of the approximation

The EKF retains a Gaussian after using a tangent approximation. Its accuracy therefore depends on:

- **The function's curvature.** A nearly linear function is well represented by its tangent near the mean.
- **The belief's width.** A broad belief covers more of the function, where the tangent may be inaccurate.
- **The number of hypotheses.** One Gaussian cannot preserve several separated modes, even if the local calculations are accurate.

A mixture $\sum_j a_j \mathcal{N}(x; \mu_j, \Sigma_j)$, with $a_j \geq 0$ and $\sum_j a_j = 1$, can retain several hypotheses. The book also mentions unscented methods and moment matching as alternatives to tangent-based approximation.

3.6 The information filter

The same nonsingular Gaussian can be stored in CANONICAL FORM:

$$\Omega = \Sigma^{-1}, \qquad \xi = \Omega\mu, \qquad \Sigma = \Omega^{-1}, \qquad \mu = \Omega^{-1}\xi.$$

Ω is the INFORMATION MATRIX, also called precision; ξ is the information vector. Expanding the Gaussian exponent gives

$$p(x) \propto \exp\left[-\tfrac{1}{2}x^T\Omega x + x^T\xi\right].$$

Terms independent of x are absorbed into the normalizer. Thus multiplying Gaussian factors adds their quadratic and linear coefficients. This is the reason measurements become especially simple in information form.

The centered quadratic $(x-\mu)^T\Omega(x-\mu)$ is the squared MAHALANOBIS DISTANCE: it measures deviation relative to the uncertainty, including correlations. A displacement along a high-variance direction is less surprising than the same displacement along a tightly constrained direction.

3.6.1 Prediction and correction

Substituting $\Sigma = \Omega^{-1}$ and $\mu = \Omega^{-1}\xi$ into the Kalman prediction gives

$$\bar{\Omega}_t = (A_t\Omega_{t-1}^{-1}A_t^T + R_t)^{-1}, \tag{3.15}$$

$$\bar{\xi}_t = \bar{\Omega}_t(A_t\Omega_{t-1}^{-1}\xi_{t-1} + B_t u_t). \tag{3.16}$$

For measurement $z_t = C_t x_t + \delta_t$, correction is additive:

$$\Omega_t = \bar{\Omega}_t + C_t^T Q_t^{-1} C_t, \tag{3.17}$$

$$\xi_t = \bar{\xi}_t + C_t^T Q_t^{-1} z_t. \tag{3.18}$$

To derive these, expand the log likelihood:

$$-\tfrac{1}{2}(z-Cx)^T Q^{-1}(z-Cx) = -\tfrac{1}{2}x^T C^T Q^{-1} C x + x^T C^T Q^{-1} z + \text{constant}.$$

Adding it to the predicted Gaussian's exponent gives exactly (3.17) and (3.18). The information filter and Kalman filter therefore encode the same posterior under the same linear Gaussian assumptions; they perform the bookkeeping differently.

3.6.2 The tracking example in information form

Reuse the earlier predicted mean 12, variance 5, and reading 14 with noise variance 4. The measurement adds its contribution to each canonical parameter:

Parameter	Prediction	Measurement contribution	Posterior
Precision Ω	1/5	1/4	9/20
Information ξ	12/5	14/4	59/10

Recovering moments gives

$$\Sigma = \frac{1}{\Omega} = \frac{20}{9}, \qquad \mu = \frac{\xi}{\Omega} = \frac{118}{9}.$$

These are the Kalman filter's results. The two filters combine the same prior and likelihood, using different stored parameters.

3.7 The extended information filter

The EXTENDED INFORMATION FILTER (EIF) applies the same Taylor approximations as the EKF, while storing canonical parameters. First recover μ_{t-1} by solving $\Omega_{t-1}\mu_{t-1} = \xi_{t-1}$. Prediction is

$$\begin{aligned}
\bar{\mu}_t &= g(u_t, \mu_{t-1}), \\
\bar{\Omega}_t &= (G_t \Omega_{t-1}^{-1} G_t^T + R_t)^{-1}, \\
\bar{\xi}_t &= \bar{\Omega}_t \bar{\mu}_t.
\end{aligned}$$

The important detail is the measurement's affine offset. Linearization gives

$$z_t \approx h(\bar{\mu}_t) + H_t(x_t - \bar{\mu}_t) + \delta_t.$$

Move the known offset to the measurement side:

$$\widetilde{z}_t := z_t - h(\bar{\mu}_t) + H_t \bar{\mu}_t \quad \Longrightarrow \quad \widetilde{z}_t \approx H_t x_t + \delta_t.$$

Now apply the linear information update:

$$\Omega_t = \bar{\Omega}_t + H_t^T Q_t^{-1} H_t, \tag{3.19}$$

$$\xi_t = \bar{\xi}_t + H_t^T Q_t^{-1} [z_t - h(\bar{\mu}_t) + H_t \bar{\mu}_t]. \tag{3.20}$$

The offset contains $+H_t\bar{\mu}_t$ because it was moved from the state side to the measurement side. For a linear model $h(x) = Cx$, the offset cancels and $\widetilde{z} = z$. Using the same linearization as the EKF gives the same Gaussian approximation.

3.8 Computational and modeling tradeoffs

Moment and information forms have complementary strengths:

Operation	Moment form	Information form
Motion prediction	Direct mean/covariance propagation	Generally needs state-sized solves or inversions
Measurement update	Gain uses a measurement-sized solve	Add likelihood information to Ω and ξ
Retrieve mean	Already stored	Solve $\Omega\mu = \xi$

The main tradeoffs are:

- **Computation.** With standard dense algebra, general covariance prediction can take $O(d^3)$ time. A measurement update takes $O(d^2k + dk^2 + k^3)$, where k is measurement dimension. Small measurements and structured matrices can reduce the work.
- **Uncertainty.** Zero precision formally represents no information in a direction. A completely flat prior on an unbounded space is improper, and its mean cannot be recovered by the inverse formulas until sufficient information is available.
- **Combining measurements.** Independent likelihood contributions for the same state add in information form. Shared prior information must not be added repeatedly.
- **Sparsity.** Local interactions can make the information matrix sparse even when the covariance is dense. Exploiting that structure can make large problems more efficient.

Neither representation is uniformly preferable. The useful choice depends on the state size, measurement size, and structure of the model.

Source. Thrun, Burgard, and Fox, Probabilistic Robotics, early draft, Chapter 3, pp. 33–66 (PDF pp. 45–78).

Chapter 4: Nonparametric Filters

A Gaussian commits to one particular shape. Nonparametric filters instead describe a belief using cells or samples, and can represent several separated hypotheses. "Nonparametric" does not mean that nothing is stored: the resolution or number of samples is adjustable, rather than fixing the belief to a small family of shapes.

4.1 Discrete Bayes filters

For states $s_1, \dots, s_K$, store the probability vector $p_t = (p_{1,t}, \dots, p_{K,t})^T$. Define

$$T_{ki}(u_t) = P(X_t = s_k \mid X_{t-1} = s_i, u_t), \qquad L_k = p(z_t \mid X_t = s_k).$$

Columns of T sum to one: each column describes all destinations from one previous state. The Bayes recursion is

$$\bar{p}_{k,t} = \sum_{i=1}^{K} T_{ki}(u_t) p_{i,t-1}, \tag{4.1}$$

$$p_{k,t} = \frac{L_k \bar{p}_{k,t}}{\sum_{j=1}^{K} L_j \bar{p}_{j,t}}. \tag{4.2}$$

This is the finite-state form of the Bayes filter. The door calculation in Chapter 2 is the case $K = 2$. Each prediction averages transitions from all previous states; the measurement step then reweights the destinations.

Compute the prediction from the entire previous vector before replacing it with the new one. A dense transition takes $O(K^2)$ work; local transitions can be stored and processed sparsely.

The belief itself takes $O(K)$ space. If each state has at most r possible destinations, sparse prediction uses $O(Kr)$ transition contributions.

4.2 Histograms for continuous states

A histogram filter partitions a continuous state space into nonoverlapping cells $C_1, \dots, C_K$. Store cell *mass* $p_k = P(X \in C_k)$, not a density value. With cell volume $v_k = |C_k|$, the piecewise-uniform approximation is

$$b(x) = \frac{p_k}{v_k} \qquad \text{for } x \in C_k.$$

Integrating over a cell recovers its mass p_k. The density and the mass must therefore be kept distinct, especially when cell sizes differ.

For example, partition a uniform belief on $[0, 3]$ into $C_1 = [0, 1)$ and $C_2 = [1, 3]$. The density is $1/3$ throughout the interval:

	C_1	C_2
cell width v_k	1	2
density	$1/3$	$1/3$
mass p_k	$1/3$	$2/3$

The wider cell has twice the mass because it covers twice the interval, not because its density is larger. The two masses sum to one.

4.2.1 Convert continuous models to cell models

Under the piecewise-uniform approximation, average the likelihood within a cell, and integrate motion over source and destination cells:

$$L_k = \frac{1}{v_k} \int_{C_k} p(z_t \mid x)\, dx,$$
$$T_{ki} = \frac{1}{v_i} \int_{C_i} \int_{C_k} p(x' \mid u_t, x)\, dx'\, dx.$$

Then use the discrete recursion (4.2). These integrals describe the cell model exactly under the uniform-within-cell assumption; they are not generally the exact continuous posterior after an update.

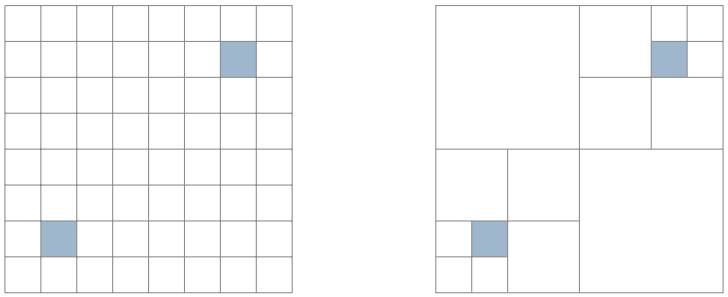
If the cells are small and the models vary little within them, representative centers c_i, c_k give the midpoint approximations

$$L_k \approx p(z_t \mid c_k), \qquad T_{ki} \approx v_k\, p(c_k \mid u_t, c_i).$$

The destination volume *multiplies* the transition density to produce probability mass. Equal volumes can cancel in a common normalization, but unequal volumes cannot be discarded. Boundary handling and numerical approximation must preserve valid transition probabilities; silently losing mass outside a modeled region changes the model.

4.2.2 Where to spend resolution

A fixed grid is straightforward, but resolution is expensive. Ten cells per coordinate require 10^d cells in d dimensions. Refining every region equally spends work on locations that may have almost no probability.



fixed grid adaptive partition

Figure 5: Schematic partitions for two regions of high belief. An adaptive tree retains fine cells near those regions and uses larger cells elsewhere. Shading identifies the regions, rather than specifying numerical masses.

There are several ways to allocate the stored values:

- **Static grids** choose the partition in advance. They are simple, but use the same resolution in likely and unlikely regions.
- **Density trees** refine the partition according to the belief, using smaller cells where more detail is useful.
- **Selective updating** retains a grid but updates only a selected subset of its cells, representing the remaining region more coarsely.

Adaptive methods can save substantial computation, with more complicated bookkeeping. Completely dropping an unlikely region can also discard a hypothesis needed later.

Topological representations use significant places, such as junctions, as states. Metric representations retain numerical spatial relationships. This choice of spatial model is separate from whether the belief's partition is fixed or adaptive.

4.3 Static binary state and log odds

Suppose an unknown binary fact x does not change, and successive readings are conditionally independent given that fact. An example is whether a fixed map cell is occupied, under the chosen static-world model. Let $p_t = P(x \mid z_{1:t})$ and define the LOG ODDS

$$\ell_t = \log \frac{p_t}{1 - p_t}, \qquad p_t = \frac{1}{1 + \exp(-\ell_t)}.$$

Use natural logarithms consistently here. Probability $1/2$ corresponds to zero log odds; positive values favor x and negative values favor $\neg x$. The initial value is

$$\ell_0 = \log \frac{p_0}{1 - p_0}.$$

Dividing the Bayes update for x by the update for $\neg x$ cancels the normalizer:

$$\frac{p_t}{1-p_t} = \frac{p(z_t \mid x)}{p(z_t \mid \neg x)} \frac{p_{t-1}}{1-p_{t-1}}.$$

Taking logarithms turns repeated likelihood multiplication into addition.

Often an INVERSE MEASUREMENT MODEL directly supplies $q_t = P(x \mid z_t)$, computed using prior p_0, instead of describing all possible measurements through $p(z_t \mid x)$. Applying Bayes' rule once to this single-reading posterior gives

$$\frac{p(z_t \mid x)}{p(z_t \mid \neg x)} = \frac{q_t}{1-q_t} \frac{1-p_0}{p_0}.$$

Consequently,

$$\ell_t = \ell_{t-1} + \log \frac{q_t}{1-q_t} - \log \frac{p_0}{1-p_0}. \tag{4.3}$$

Subtracting the prior's log odds prevents counting that prior again with every inverse-model output. The inverse model must use the same reference prior assumed in this conversion.

4.3.1 Example using the door sensor

Keep the door unchanged and take two camera reports, both saying "open". Use the sensor from Chapter 2:

$$p(\text{report open} \mid O) = 0.6, \qquad p(\text{report open} \mid C) = 0.2.$$

Start with $p_0 = P(O) = 1/2$, so the initial odds are one. Each report multiplies the odds by the likelihood ratio $0.6/0.2 = 3$:

	initial	one report	two reports
odds for open	1	3	9
probability open	1/2	3/4	9/10

In log form, each report adds $\log 3$. The single-reading inverse model is $q_t = 3/4$ under prior $1/2$, so it produces the same increment in (4.3).

The second independent report strengthens the evidence that the door is open. This example assumes the state stays fixed. A push, as in Chapter 2, requires a transition prediction before the next reading. Reprocessing the same image would also not count as a new independent report.

4.4 Particle beliefs and the bootstrap filter

A PARTICLE is one possible value of the state. A weighted set $\{(x^{[m]}, w^{[m]})\}_{m=1}^{M}$, with nonnegative weights summing to one, approximates expectations and region probabilities:

$$\mathbb{E}[\varphi(X)] \approx \sum_{m=1}^{M} w^{[m]} \varphi(x^{[m]}), \qquad P(X \in A) \approx \sum_{m=1}^{M} w^{[m]} \mathbf{1}\{x^{[m]} \in A\}.$$

The empirical distribution places mass at the particle locations; it is not an ordinary smooth density. Several clusters can preserve several location hypotheses. With equal weights, probability in a region is approximately the fraction of particles there.

The BOOTSTRAP PARTICLE FILTER starts with samples from $p(x_0)$. At each step it simulates motion, scores the resulting hypotheses against the measurement, then reallocates samples through RESAMPLING.

Algorithm 4.1: Bootstrap particle-filter update

Input: Equal-weight particles $\{x_{t-1}^{[m]}\}_{m=1}^{M}$, u_t, z_t
for $m = 1, \dots, M$ **do**
  $\widetilde{x}_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]})$
  $\widetilde{w}_t^{[m]} \leftarrow p(z_t \mid \widetilde{x}_t^{[m]})$
Normalize the weights: $w_t^{[m]} \leftarrow \widetilde{w}_t^{[m]} / \sum_j \widetilde{w}_t^{[j]}$
for $m = 1, \dots, M$ **do**
  Draw an index i_m with probability $w_t^{[i_m]}$
  $x_t^{[m]} \leftarrow \widetilde{x}_t^{[i_m]}$
return $\{x_t^{[m]}\}_{m=1}^{M}$, each with weight $1/M$

The draws in the last loop are with replacement, so a strong hypothesis can produce several copies. Resampling relocates computational effort toward likely states; it does not create information or make the approximation exact. The weighted set *before* resampling already represents the measurement update and can be used to estimate expectations directly.

Implementation requires two operations: sample from the transition model, and evaluate the measurement likelihood at a particle. The weights must have positive total before they can be normalized.

4.5 Why importance weights work

Suppose the desired TARGET DENSITY is f, but samples can only be drawn from a PROPOSAL DENSITY g. Assign each sample weight

$$\widetilde{w}^{[m]} = \frac{f(x^{[m]})}{g(x^{[m]})}, \qquad x^{[m]} \sim g.$$

The proposal must cover the target: $f(x) > 0$ requires $g(x) > 0$. Otherwise some target regions can never be represented, regardless of how weights are adjusted. The reason for the ratio is cancellation inside an expectation:

$$\mathbb{E}_g\left[\frac{f(X)}{g(X)}\varphi(X)\right] = \int g(x)\frac{f(x)}{g(x)}\varphi(x)\,dx = \mathbb{E}_f[\varphi(X)].$$

If f is known only up to a constant, normalize the sampled weights:

$$\mathbb{E}_f[\varphi(X)] \approx \frac{\sum_m \widetilde{w}^{[m]}\varphi(x^{[m]})}{\sum_m \widetilde{w}^{[m]}}.$$

The finite-sample ratio is generally biased. Under suitable finite-variance conditions, errors in these estimates decrease on the order of $M^{-1/2}$. A proposal that poorly covers important parts of the target can still require many particles. The approximation concerns expectations and region probabilities; the particles themselves remain a discrete distribution.

4.5.1 Why bootstrap weights are just likelihoods

For the derivation, temporarily consider a whole state path $x_{0:t}$. The Markov model gives the target factorization

$$\begin{aligned} p(x_{0:t} \mid z_{1:t}, u_{1:t}) \propto\; & p(z_t \mid x_t)\, p(x_t \mid x_{t-1}, u_t) \\ & \cdot\, p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t-1}). \end{aligned}$$

If the previous paths already represent the preceding posterior and we extend them by sampling the motion model, the proposal contains the last two factors. Target divided by proposal therefore leaves only

$$\widetilde{w}_t \propto p(z_t \mid x_t).$$

Keeping the current endpoints of the weighted paths gives a representation of the current-state posterior. This explains the bootstrap algorithm's weight, while its finite particle set still introduces approximation error.

If resampling is skipped, previous particles are not reset to equal weights. Their weights must instead be propagated multiplicatively:

$$\widetilde{w}_t^{[m]} = w_{t-1}^{[m]} p(z_t \mid \widetilde{x}_t^{[m]}).$$

Reset to $1/M$ only *after* resampling. Resampling every step is a simple baseline, rather than a requirement of Bayes filtering.

4.6 Systematic resampling

Independent resampling can waste samples by chance. SYSTEMATIC RESAMPLING, called low-variance resampling in the book, uses one random offset and evenly spaced selection points. It operates on normalized weights.

Algorithm 4.2: Systematic resampling

Input: Particles $\{x^{[i]}\}_{i=1}^{M}$ and weights $w^{[i]}$ summing to one

Draw $r \sim \text{Uniform}[0, 1/M)$
$i \leftarrow 1$; $c \leftarrow w^{[1]}$
for $m = 1, \ldots, M$ **do**
    $u \leftarrow r + (m-1)/M$
    while $u \geq c$ **do**
        $i \leftarrow i + 1$
        $c \leftarrow c + w^{[i]}$
    Output a copy of $x^{[i]}$ with weight $1/M$

Each cumulative-weight interval has length $w^{[i]}$. The algorithm copies the particle whose interval contains the selection point. Choosing the first cumulative value greater than u also skips zero-weight intervals.

The pointer i only moves forward and crosses the list at most once, so the whole operation takes $O(M)$ time. A particle's expected number of copies is $Mw^{[i]}$. With equal weights, every particle is copied once, avoiding random loss of diversity in that special case.

4.6.1 Worked example: turning weights into copies

Suppose the motion and measurement steps have produced four weighted location hypotheses. We now want four equal-weight particles representing the same belief approximately. The example begins *after* likelihood evaluation; the location values and normalized weights are illustrative.

particle index	1	2	3	4
location	1	2	5	6
weight	0.1	0.2	0.6	0.1
cumulative weight	0.1	0.3	0.9	1

1. **Make cumulative intervals.** Particle 1 occupies $[0, 0.1)$, particle 2 occupies $[0.1, 0.3)$, particle 3 occupies $[0.3, 0.9)$, and particle 4 occupies $[0.9, 1)$.

2. **Choose equally spaced points.** With $M = 4$, spacing is $1/4$. Choose the possible random offset $r = 0.08$. The points are $0.08, 0.33, 0.58, 0.83$.

3. **Copy the selected particles.**

Point	Interval / particle index	Copied location
0.08	1	1
0.33	3	5
0.58	3	5
0.83	3	5

The new set is $(1, 5, 5, 5)$, each with weight $1/4$. Location 5 receives several copies because its interval is large. A different random offset can give different counts; resampling is an approximation to the weighted set.

In the diagram, horizontal distance represents cumulative probability, not physical location. The labels *inside* the intervals identify the location hypotheses.

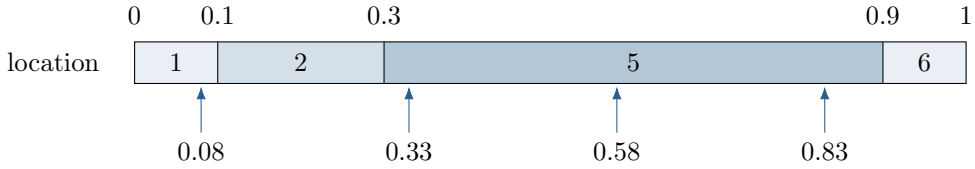


Figure 6: Interval widths are particle weights. Equally spaced cumulative probability points select particles to copy.

4.7 Failure modes and resource choices

The book separates four sources of approximation error:

1. **Finite sample size.** With $M = 1$ and positive likelihood, the only weight normalizes to one. The filter keeps that one proposed state regardless of the current reading. A finite particle set therefore does not give exact posterior sampling.
2. **Resampling variance.** For a static state with no informative readings, repeated independent resampling can eventually leave copies of only one initial hypothesis. Diversity has disappeared without information having arrived. Resampling less often or using a systematic scheme can reduce this error. Resampling too rarely, however, leaves many particles with negligible weights.
3. **Proposal–target mismatch.** A motion proposal may spread particles widely while a precise measurement favors a small region. Most weights then become negligible. An exact lower-dimensional measurement constraint may be missed entirely. Proposals informed by measurements can improve coverage of the target.
4. **Particle deprivation.** All particles near a relevant state or mode may be lost. Weighting the remaining particles cannot recreate that hypothesis. Additional recovery samples can restore coverage, but alter the approximation unless their proposal is accounted for.

The particle count depends on state dimension, uncertainty, and the number of plausible hypotheses. Adaptive methods change it as the belief changes; resource-adaptive methods also respond to the computation available.

Source. Thrun, Burgard, and Fox, Probabilistic Robotics, early draft, Chapter 4, pp. 67–90 (PDF pp. 79–102).